JUINO MEDEIROS

CHATBOT HÍBRIDO PARA O EDUCA

Arquitetura e funcionamento do protótipo

Natal
2026

JUINO MEDEIROS

CHATBOT HÍBRIDO PARA O EDUCA

Arquitetura e funcionamento do protótipo

Relatório técnico de documentação do protótipo de atendimento para o Educa no Campus de Natal, com descrição da extensão para WhatsApp Web, da API, da recuperação de informações e do processamento de áudio.

Natal
2026

RESUMO

O atendimento pelo WhatsApp permite que o público procure informações por texto e por mensagens de voz. Este relatório descreve um protótipo de chatbot para o Educa no Campus de Natal, desenvolvido com uma extensão para o navegador Brave e uma API em Python. O atendimento combina regras para informações previamente verificadas com recuperação de trechos de documentos oficiais e geração de respostas pelo modelo Phi-4 Mini, executado por meio do Ollama. A base reúne o edital de matrícula do semestre 2026.2 e notícias do portal da Universidade do Estado do Rio Grande do Norte. A busca utiliza BM25, enquanto os dados das turmas são organizados separadamente para evitar confusão com notícias de outros períodos. Mensagens de voz são transcritas localmente pelo Whisper, e as respostas faladas são geradas pela voz Francisca, acessada pelo serviço online do Microsoft Edge. Os testes verificaram respostas combinadas, histórico por conversa, formato de áudio e execução do modelo na GPU. O protótipo fornece orientação, mas não realiza matrícula, não consulta vagas em tempo real e não encaminha automaticamente a conversa para um atendente. Sua utilização institucional depende de avaliação do funcionamento na sessão do WhatsApp e de acompanhamento das informações da base.

Palavras-chave: chatbot; WhatsApp; recuperação de informações; modelo de linguagem; atendimento.

SUMÁRIO

1 INTRODUÇÃO 5

2 ARQUITETURA E TECNOLOGIAS 6

3 FUNCIONAMENTO DA EXTENSÃO 7

4 RECUPERAÇÃO DE INFORMAÇÕES 8

5 PROCESSAMENTO DE ÁUDIO 9

6 HISTÓRICO E CONTROLE DO ATENDIMENTO 10

7 VALIDAÇÃO E OPERAÇÃO 11

8 CONSIDERAÇÕES FINAIS 12

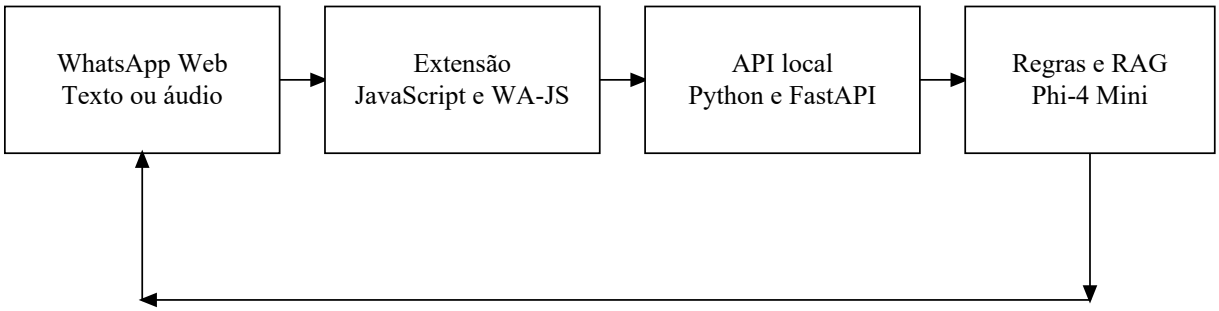
REFERÊNCIAS 13

1 INTRODUÇÃO

O Educa oferece cursos e atividades de extensão no Campus de Natal da Universidade do Estado do Rio Grande do Norte. As dúvidas do público envolvem, entre outros assuntos, faixa de idade, horário das aulas, gratuidade e matrícula. O edital 019/2026 reúne as condições previstas para o semestre 2026.2 (FUNCITERN; UERN, 2026).

O protótipo documentado neste relatório foi desenvolvido para responder a essas perguntas em uma conversa do WhatsApp Web. Sua estrutura reúne uma extensão no navegador Brave e uma API executada no computador. A extensão recebe as mensagens e envia as respostas; a API consulta a base, aplica regras e, quando necessário, solicita uma resposta ao modelo de linguagem.

Figura 1 - Arquitetura geral do chatbot



Fonte: elaboração própria (2026).

A Figura 1 apresenta o caminho de entrada e saída das mensagens. O atendimento aceita texto e áudio, mantém um histórico curto por conversa e combina respostas controladas com recuperação de informações. O uso de áudio foi incluído para permitir que a pessoa escolha como se comunicar, sem pressupor dificuldades de leitura ou escrita.

O sistema tem função de orientação. Ele não realiza matrícula, não confirma vagas disponíveis e não atende automaticamente todas as conversas. A ativação ocorre em uma conversa individual escolhida no painel da extensão, inclusive no modo de teste em “Mensagem para você”.

2 ARQUITETURA E TECNOLOGIAS

A extensão foi escrita em JavaScript, com interface em HTML e CSS. A API utiliza Python e FastAPI, que organiza os endpoints de acesso ao processamento (FastAPI, s.d.). O Quadro 1 relaciona os componentes usados na versão documentada.

Quadro 1 - Componentes e tecnologias do protótipo

Componente	Tecnologia	Função
Extensão	JavaScript	Receber mensagens e controlar a comunicação com a API.
Interface	HTML e CSS	Exibir ativação, pausa e andamento do atendimento.
WhatsApp Web	WA-JS 4.6.1	Consultar mensagens e enviar texto ou áudio.
API	Python e FastAPI	Aplicar regras e organizar busca, histórico e áudio.
Modelo	Phi-4 Mini e Ollama	Gerar respostas com apoio dos trechos recuperados.
Busca	rank-bm25	Classificar os trechos por relevância lexical.
Extração	pypdf e BeautifulSoup	Ler o edital e extrair o conteúdo do portal.
Persistência	JSON e SQLite	Guardar a base, respostas e histórico curto.
Transcrição	faster-whisper small	Converter o áudio recebido em texto na CPU.
Voz	edge-tts e Francisca	Gerar a resposta falada no serviço da Microsoft.
Conversão	PyAV e Opus	Preparar áudio em OGG/Opus para o envio.

Fonte: elaboração própria (2026).

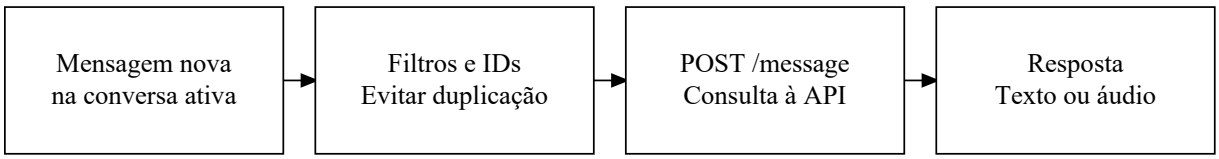
A comunicação entre extensão e API utiliza requisições HTTP com dados em JSON. JavaScript e Python executam a lógica do sistema; HTML e CSS definem a apresentação do painel. Os arquivos JSON armazenam documentos e dados de turmas, enquanto SQLite registra o histórico e as respostas em cache.

A voz atual depende de um serviço online. O modelo de linguagem, a busca e a transcrição são executados no computador. Essa distinção é importante porque a geração da fala envia o texto preparado da resposta à Microsoft (Rany2, s.d.).

3 FUNCIONAMENTO DA EXTENSÃO

Quando o usuário ativa o atendimento, a extensão verifica a API e identifica a conversa aberta. As mensagens já existentes são registradas como ponto de partida. Novas mensagens são verificadas aproximadamente a cada cinco segundos, embora o navegador possa atrasar essa frequência em segundo plano. A leitura e o envio utilizam WA-JS, biblioteca que opera sobre a sessão do WhatsApp Web (WPPConnect, s.d.).

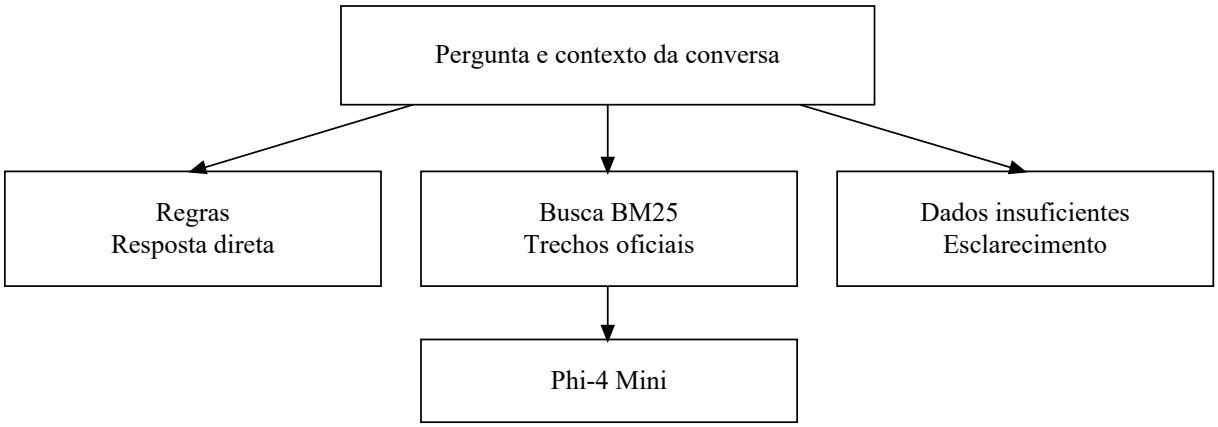
Figura 2 - Processamento de uma mensagem pela extensão



Fonte: elaboração própria (2026).

O bot é híbrido: informações verificadas podem receber uma resposta direta por regras, enquanto outras perguntas exigem consulta aos documentos e uso da Phi. Uma mesma mensagem pode combinar assuntos, como gratuidade e faixa de idade. Nesse caso, o sistema reúne as informações necessárias para responder às partes reconhecidas da pergunta.

Figura 3 - Decisão entre regras e modelo de linguagem



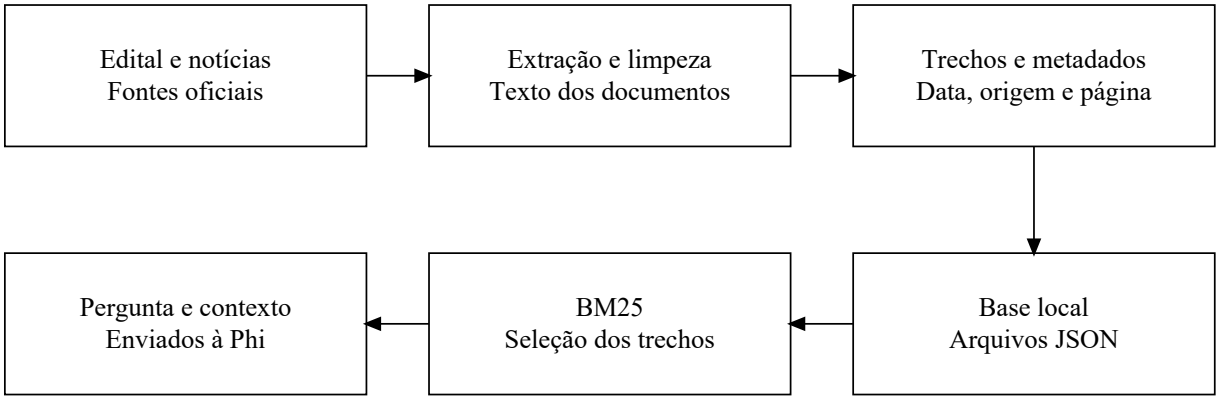
Fonte: elaboração própria (2026).

A resposta deve se limitar ao pedido atual. Uma pergunta sobre dia e horário não exige repetir idade, documentos ou disponibilidade de vagas. O histórico ajuda a identificar a turma mencionada anteriormente, mas não deve ampliar o assunto por conta própria.

4 RECUPERAÇÃO DE INFORMAÇÕES

A recuperação de informações usada no projeto é conhecida como RAG. Em vez de treinar a Phi com o edital, a API seleciona trechos relacionados à pergunta e os envia ao modelo. A busca utiliza BM25, implementação disponível na biblioteca rank-bm25 (Brown, s.d.).

Figura 4 - Preparação da base e recuperação dos trechos



Fonte: elaboração própria (2026).

A ingestão registrada em 8 de outubro de 2026 reúne 17 documentos, correspondentes a sete páginas do edital e dez notícias do portal. A divisão produziu 34 trechos. A tabela do edital foi organizada separadamente em 20 registros de turmas, adicionados à recuperação pela API. Assim, a versão documentada pesquisa 54 itens.

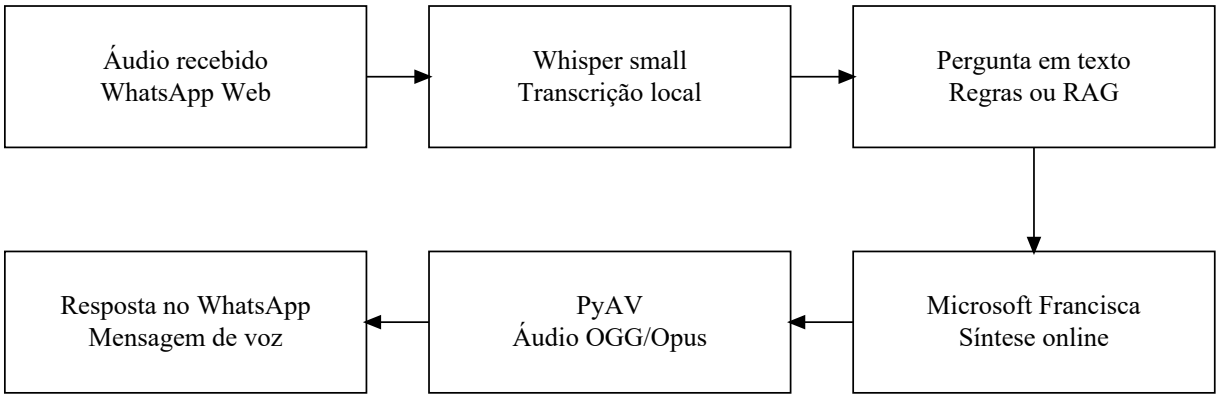
A busca é lexical, baseada nas palavras da pergunta, sem embeddings ou banco vetorial. Os trechos preservam título, origem, data e, quando disponível, página. Para horário e idade das turmas, a API prioriza os dados da tabela, evitando misturar o edital com notícias de outros períodos.

A base não cobre todo o histórico do portal e não é atualizada durante cada atendimento. Novos documentos precisam passar pela ingestão, seguida de reinício da API. O edital previa início das aulas em 3 de agosto de 2026; essa previsão não comprova o início efetivo nem a abertura de novas inscrições (FUNCITERN; UERN, 2026).

5 PROCESSAMENTO DE ÁUDIO

O processamento começa quando a extensão identifica uma mensagem de voz na conversa ativada. O áudio é baixado e enviado à API, que utiliza faster-whisper para obter a transcrição. Essa etapa ocorre no computador, na CPU (SYSTRAN, s.d.).

Figura 5 - Transcrição da entrada e geração da resposta falada



Fonte: elaboração própria (2026).

Depois de preparar a resposta, a API envia seu texto ao serviço de voz do Microsoft Edge, usando a biblioteca comunitária edge-tts. A voz configurada é pt-BR-FranciscaNeural. A biblioteca não exige instalar o navegador Edge nem fornecer uma chave de API, mas depende de internet e da disponibilidade do serviço (Rany2, s.d.).

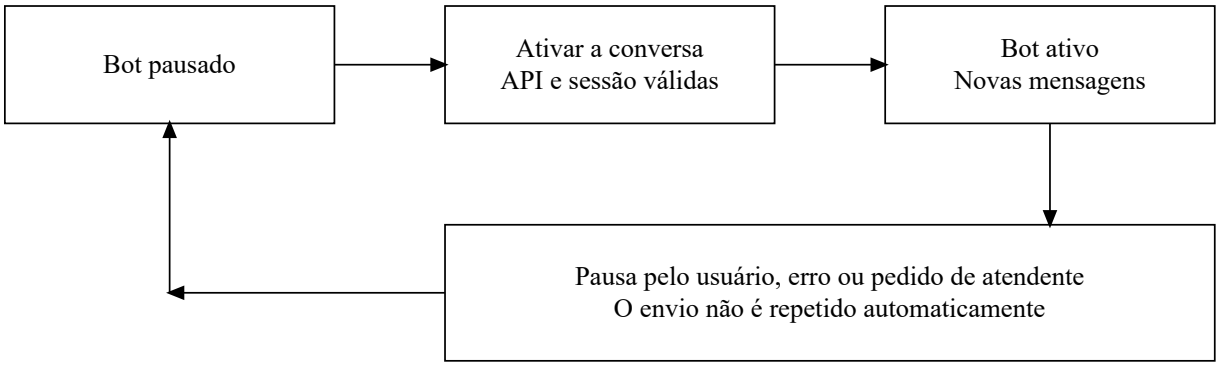
O áudio gerado é convertido para OGG/Opus e enviado como mensagem de voz. Antes da síntese, o texto passa por ajustes para retirar emojis e preparar a leitura de horários. As grafias Educa, EdUCA e EDUCA são convertidas para “educa” somente no texto falado, para reduzir a leitura do nome como sigla.

O protótipo aceita áudios de até três minutos e 12 MB. Ruído, sotaque, nomes e números podem prejudicar a transcrição. Quando não entende o conteúdo, o bot pede uma repetição. Essas condições precisam ser avaliadas com mensagens reais antes de ampliar o atendimento.

6 HISTÓRICO E CONTROLE DO ATENDIMENTO

A extensão atende apenas a conversa ativada e registra os identificadores das mensagens já processadas. As saídas do próprio bot também são identificadas para evitar que ele responda a si mesmo. Um resultado de envio incerto não é repetido automaticamente.

Figura 6 - Ativação e pausa do atendimento



Fonte: elaboração própria (2026).

A API conserva até quatro pares de pergunta e resposta por conversa. As últimas duas trocas podem ser enviadas à Phi como contexto. Isso permite compreender perguntas como “E qual o horário?” depois de uma mensagem que identifica a turma. O histórico é separado por conversa e não é usado para treinar o modelo.

O acesso à API ocorre em 127.0.0.1:8765 e exige a chave local no cabeçalho X-Educa-Token. A chave fica nos arquivos de configuração da API e da extensão. Os registros são guardados em SQLite, com política de retenção de 24 horas e limpeza na inicialização e no processamento das mensagens. Perguntas e respostas permanecem legíveis no banco local; os áudios recebidos são processados em memória.

Quando o usuário pede um atendente, o bot envia a resposta de pausa e interrompe novas mensagens automáticas. Essa ação não notifica uma pessoa nem atribui a conversa a alguém da equipe. A extensão também pode ser pausada pelo painel ou interrompida por erro.

A integração utiliza a sessão do WhatsApp Web e não a API oficial de atendimento. Mudanças na plataforma podem afetar o funcionamento, e existe risco de restrição da conta. O piloto precisa de acompanhamento antes de uso institucional.

7 VALIDAÇÃO E OPERAÇÃO

Os testes locais verificaram o processamento da API, a aplicação de regras e a recuperação de informações. Foram usados casos com perguntas combinadas, continuação da conversa e solicitação de apenas um dado, como dia ou horário. O Quadro 2 apresenta o alcance dessas verificações.

Quadro 2 - Verificações realizadas no protótipo

Verificação	Resultado e alcance
API e regras	Autenticação, gratuidade, faixas de idade e perguntas combinadas verificadas localmente.
Histórico	Continuidade, isolamento entre conversas, limite de quatro trocas e repetição da mesma requisição.
Áudio	Síntese Microsoft, leitura de arquivos OGG/Opus e transcrição verificadas nos componentes locais.
WhatsApp real	Respostas em texto e comportamento da voz relatados pelo usuário; a sessão real não é inteiramente coberta pelos testes locais.

Fonte: elaboração própria (2026).

Na máquina utilizada, a GTX 1650 possui 4 GB de memória. A configuração testada carregou a Phi-4 Mini inteira na GPU, mantendo contexto de 3.072 tokens e ocupação de aproximadamente 2,9 GB na medição do driver. O indicador 100% GPU do Ollama corresponde ao carregamento integral do modelo nessa memória (Ollama, s.d.). A transcrição permaneceu na CPU.

Uma comparação curta de geração registrou cerca de 20 tokens por segundo no modo misto CPU/GPU e 41 tokens por segundo com o modelo inteiro na GPU. O resultado descreve aquele teste e não garante a mesma latência para mensagens longas. Respostas resolvidas por regras não dependem da execução da Phi.

Para atender, o computador, o Ollama, a API e o WhatsApp Web precisam estar disponíveis. A API é iniciada por INICIAR.cmd, e a conversa é ativada no painel da extensão. Pausar a extensão interrompe as respostas automáticas; encerrar a API desliga o processamento.

8 CONSIDERAÇÕES FINAIS

O protótipo reúne, em uma mesma conversa, respostas por regras, consulta a documentos e comunicação por áudio. A extensão permite aproveitar a sessão do WhatsApp Web, enquanto a API concentra a base e o processamento. Essa organização facilita a revisão das respostas e a atualização das fontes sem alterar toda a interface do navegador.

As regras são úteis para informações que podem ser conferidas diretamente, como gratuidade, faixa de idade e horário de uma turma. A recuperação de trechos amplia o conjunto de perguntas que pode ser respondido, mas o modelo ainda pode acrescentar detalhes desnecessários ou interpretar uma fonte de maneira inadequada. Por isso, as respostas precisam ser limitadas ao pedido atual e avaliadas com exemplos de uso real.

A disponibilidade de vagas e as datas de novas inscrições continuam dependendo de confirmação pela equipe. Uma previsão histórica do edital não permite anunciar inscrições abertas. Da mesma forma, a oferta de atendimento humano precisa corresponder a uma ação que o sistema realiza: nesta versão, o bot apenas pausa suas respostas, sem chamar um atendente automaticamente.

Antes de ampliar o piloto, é necessário acompanhar o funcionamento na sessão do Brave, revisar os resultados da transcrição e manter a base atualizada. Também devem ser avaliados o acesso aos registros locais e o envio do texto da resposta ao serviço de voz. Essas verificações ajudam a estabelecer o que o chatbot pode responder sozinho e o que deve permanecer sob responsabilidade da equipe.

REFERÊNCIAS

BROWN, Dorian. **rank_bm25: a collection of BM25 algorithms in Python**. [S. l.], [s. d.]. Disponível em: https://github.com/dorianbrown/rank_bm25. Acesso em: 8 out. 2026.

FASTAPI. **FastAPI**. [S. l.], [s. d.]. Disponível em: https://fastapi.tiangolo.com/. Acesso em: 8 out. 2026.

FUNCITERN; UNIVERSIDADE DO ESTADO DO RIO GRANDE DO NORTE. **Edital nº 019/2026 FUNCITERN PROEX UERN: matrículas de alunos para o semestre 2026.2**. Natal, 23 jul. 2026. 7 p. Documento fornecido para a base do protótipo.

OLLAMA. **FAQ**. [S. l.], [s. d.]. Disponível em: https://docs.ollama.com/faq. Acesso em: 8 out. 2026.

RANY2. **edge-tts**. [S. l.], [s. d.]. Disponível em: https://github.com/rany2/edge-tts. Acesso em: 8 out. 2026.

SYSTRAN. **faster-whisper: faster Whisper transcription with CTranslate2**. [S. l.], [s. d.]. Disponível em: https://github.com/SYSTRAN/faster-whisper. Acesso em: 8 out. 2026.

UNIVERSIDADE DO ESTADO DO RIO GRANDE DO NORTE. **Portal UERN: notícias da tag Educa**. [S. l.], [s. d.]. Disponível em: https://portal.uern.br/blog/tag/educa/. Acesso em: 8 out. 2026.

WPPCONNECT. **WA-JS: documentação da versão 4.6.1**. [S. l.], [s. d.]. Disponível em: https://wppconnect.io/wa-js/. Acesso em: 8 out. 2026.